

AI助手模块 (AI) 详细报告

一、需求与设计

1.1 功能需求

需求编号	需求描述	优先级
AI-01	群聊中 @AI助手 触发AI回复	P1
AI-02	AI回复基于最近30条群聊上下文	P1
AI-03	DeepSeek API 集成 (OpenAI兼容)	P1
AI-04	AI助手可配置名称和模型	P2
AI-05	API不可用时优雅降级	P1

1.2 设计决策

- 仅群聊触发 — `shouldTriggerAssistant()` 只在 GROUP 类型会话中响应
- 显式@提及触发 — 用户必须输入 `@助手名称` 才会触发, 避免无意义调用
- 上下文窗口 — 最近30条消息作为上下文发送给AI
- 开发模式降级 — `deepseek.apiKey` 为空时, AI返回预设提示信息而非报错
- HTTP超时 — 45秒超时, 防止阻塞聊天

二、后端设计

2.1 核心文件清单

文件	行数	位置
AiAssistantService.java	59	<code>src/main/java/com/example/chat/ai/AiAssistantService.java</code>
DeepSeekClient.java	123	<code>src/main/java/com/example/chat/ai/DeepSeekClient.java</code>

2.2 AiAssistantService — 核心业务逻辑

类结构与依赖:

```
public class AiAssistantService {
    private final String assistantName; // 默认 "千问小助手"
    private final DeepSeekClient deepSeekClient;

    // 生产构造 → 从AppConfig读取配置
    public AiAssistantService()
    // 测试构造 → 注入Mock客户端
    public AiAssistantService(String assistantName, DeepSeekClient deepSeekClient)
}
```

代码位置：src/main/java/com/example/chat/ai/AiAssistantService.java

方法签名与逻辑：

方法	功能	关键逻辑
isMentioned(String content)	检测消息是否@了助手	content.contains("@" + assistantName)
extractUserPrompt(String content)	提取@之后的问题文本	去除 @助手名 前缀，空白则返回默认提示
buildPrompt(String userPrompt, List<ChatMessage> history)	构建带上下文的提示词	拼接助手名、用户问题、最近30条消息历史
answer(String mentionMessage, List<ChatMessage> history)	完整流程入口	extractPrompt → buildPrompt → deepSeekClient.chat()

默认提示词 (无问题文本时):

```
"请根据最近聊天进行总结。"
```

系统提示词 (固定):

```
"你是一个在 Jakarta Chat 群聊中工作的中文助手。"
```

代码位置：src/main/java/com/example/chat/ai/AiAssistantService.java 第15-59行

2.3 DeepSeekClient — HTTP API 客户端

架构：

```
public class DeepSeekClient {
    private final String baseUrl; // 默认 https://api.deepseek.com
    private final String apiKey; // 从 AppConfig 读取
    private final String model; // 默认 deepseek-chat
    private final Transport transport; // HTTP客户端 (可Mock注入)
```

```
public static DeepSeekClient fromConfig() // 从 AppConfig 创建
public String chat(String systemPrompt, String userPrompt)
}
```

chat() 执行流程:

1. apiKey为空 → 返回 "DeepSeek AI 助手暂时不可用..." (不发起HTTP请求)
2. 构建 ChatRequest {model, messages: [system, user], stream: false}
3. POST JSON → <baseUrl>/v1/chat/completions
4. Bearer Auth, Content-Type JSON, 45秒超时
5. 解析 ChatResponse → 返回 choices[0].message.content.trim()
6. 任何失败 → 返回降级消息 (不抛异常)

代码位置: `src/main/java/com/example/chat/ai/DeepSeekClient.java`

内部数据结构 (Java Record):

```
record ChatRequest(String model, List<Message> messages, boolean stream)
record Message(String role, String content)
record ChatResponse(List<Choice> choices) // @JsonIgnoreProperties(ignoreUnknown =
true)
record Choice(Message message) // @JsonIgnoreProperties(ignoreUnknown =
true)
```

Testable Design — Transport 接口:

```
interface Transport {
    String postJson(Uri uri, String json, Duration timeout) throws IOException,
    InterruptedException;
}
// 生产实现: java.net.HttpClient
// 测试可注入 Mock 实现
```

代码位置: `src/main/java/com/example/chat/ai/DeepSeekClient.java` 第25-123行

三、与 Chat 模块的集成

3.1 触发流程

```
用户发送消息 → ChatService.send()
↓
ChatService.shouldTriggerAssistant(conversationType, content)
↓ (GROUP + @助手名)
ChatService.sendWithAssistantReplies(userId, sendRequest)
├─ saveMessage() — 保存用户消息
```

```
└─ aiAssistantService.answer(content, history) – 调用AI
└─ saveAssistantMessage() – 保存AI回复消息
```

代码位置：src/main/java/com/example/chat/chat/ChatService.java

- shouldTriggerAssistant() 方法 (~第520行)
- sendWithAssistantReplies() 方法 (~第130行)
- saveAssistantMessage() 方法 (~第560行)

3.2 AI 消息特征

- type = "AI" — 前端渲染为特殊颜色气泡
- senderId = aiAssistantUserId (配置项 ai.assistantUserId，默认3)
- 当 deepseek.apiKey 为空时，AI回复显示 "AI 助手当前未配置 API 密钥" 提示

3.3 配置项

配置键	默认值	说明
ai.assistantUserId	3	AI助手的系统用户ID
ai.assistantName	千问小助手	AI助手显示名称 (@触发词)
deepseek.baseUrl	https://api.deepseek.com	DeepSeek API地址
deepseek.apiKey	(空)	API密钥
deepseek.model	deepseek-chat	模型名称

代码位置：src/main/java/com/example/chat/config/AppConfig.java + 各自使用的类

四、前端设计

4.1 @提及检测

```
function isAiMention(content) {
  return content && content.includes("@千问小助手"); // 或其他配置的助手名
}
```

代码位置：src/main/webapp/assets/js/chat.js — isAiMention() (~942行)

4.2 AI思考指示器

发送@助手消息后，显示 "AI 正在思考..." 的动态指示器：

```
function showAssistantThinking() {
  // 在消息列表底部追加临时思考气泡
  // 收到AI回复后自动移除
}

function hideAssistantThinking() {
```

```
// 移除思考指示器
}
```

代码位置：src/main/webapp/assets/js/chat.js — showAssistantThinking() (~946行), hideAssistantThinking() (~964行)

4.3 AI消息渲染

AI消息 (type="AI") 在 messageBody() 中渲染为特殊样式气泡，与普通消息视觉区分。

代码位置：src/main/webapp/assets/js/chat.js — messageBody() (~692行)

五、测试覆盖

测试文件	位置	覆盖内容
AiAssistantServiceTest.java	src/test/java/com/example/chat/ai/AiAssistantServiceTest.java	触发词检测、提示构建、上下文拼接
DeepSeekClientTest.java	src/test/java/com/example/chat/ai/DeepSeekClientTest.java	API调用、降级行为、超时处理